



Visitor Pattern

1 Mục tiêu bài học

- Sau bài học này, sinh viên có thể:
- Hiểu mục đích và bản chất của Visitor Pattern
- Phân biệt Visitor với Strategy và Template Method
- Biết khi nào nên dùng Visitor
- Áp dụng Visitor vào Order Management System (OMS)
- Triển khai Visitor Pattern bằng C#

2 Vấn đề

Trong Order Management System, ta có nhiều loại đối tượng: PhysicalProduct, DigitalProduct, ServiceProduct.

Hệ thống cần thêm nhiều nghiệp vụ: Tính thuế, Phí vận chuyển, Xuất báo cáo, Khuyến mãi.

```
public class PhysicalProduct {  
    public decimal CalculateTax() { }  
    public decimal CalculateShipping() { }  
    public string GenerateReport() { }  
}
```

 Vấn đề: Class phình to (vi phạm SRP), Khó thêm nghiệp vụ mới, Phải sửa tất cả class khi thêm chức năng.

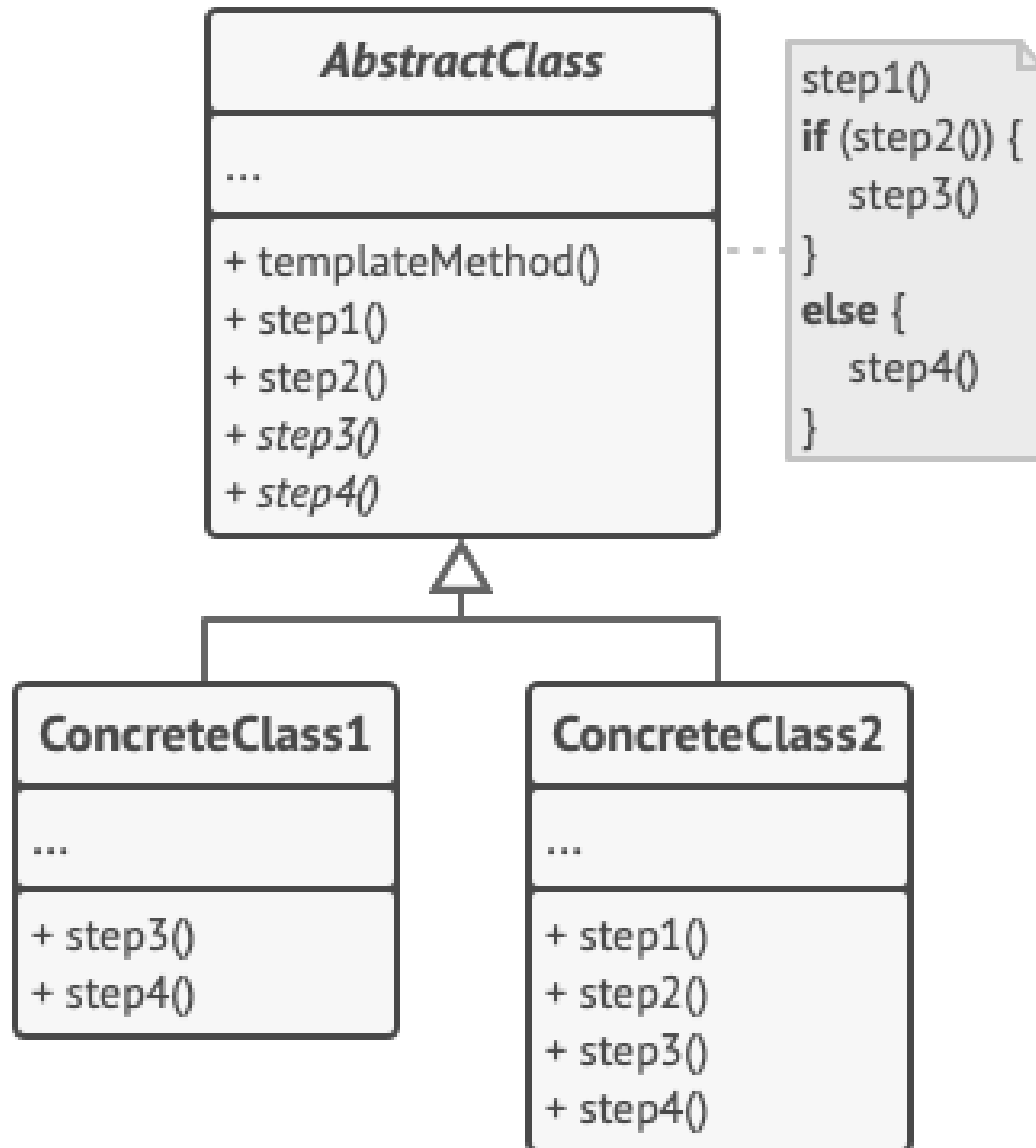
3 Giải pháp

- ❑ Giải pháp: Visitor Pattern

- ❑ Visitor cho phép thêm hành vi mới vào một tập hợp object mà không cần thay đổi class của chúng.

- ❑ Thay vì thêm method vào Product → ta tạo Visitor.

4 Structure of Visitor Pattern



5 Ví dụ 1: Tính thuế (1/2)

Bước 1: Visitor Interface

```
public interface IProductVisitor {  
    void Visit(PhysicalProduct p); void Visit(DigitalProduct  
p); void Visit(ServiceProduct p);  
}
```

Bước 2: Element Interface

```
public interface IProduct { void Accept(IProductVisitor  
visitor); }
```

Bước 3: Concrete Elements

```
public class PhysicalProduct : IProduct {  
    public void Accept(IProductVisitor visitor) {  
visitor.Visit(this); }  
}
```

5 Ví dụ 1: Ví dụ 1: Tính thuế (2/2)

Bước 4: Concrete Visitor – Tax Calculator

```
public class TaxVisitor : IProductVisitor {  
    public void Visit(PhysicalProduct p) { decimal tax = p.Price * 0.1m; }  
    public void Visit(DigitalProduct p) { decimal tax = p.Price * 0.05m; }  
}
```

Sử dụng

```
IProduct[] products = { new PhysicalProduct("Laptop", 2000), new  
DigitalProduct("E-Book", 50) };
```

```
IProductVisitor taxVisitor = new TaxVisitor();
```

```
foreach (var product in products) { product.Accept(taxVisitor); }
```

6 Ví dụ 2: Xuất báo cáo đơn hàng

Thay vì chỉnh sửa Product → tạo ReportVisitor.

```
using System;

public class ReportVisitor : IProductVisitor {
    public void Visit(PhysicalProduct p) =>
        Console.WriteLine($"Report: Physical - {p.Name}");
    public void Visit(DigitalProduct p) =>
        Console.WriteLine($"Report: Digital - {p.Name}");
    public void Visit(ServiceProduct p) =>
        Console.WriteLine($"Report: Service - {p.Name}");
}
```

👉 Thêm chức năng mới mà không sửa Product class.

7 So sánh Trước & Sau khi dùng Pattern

Không dùng Visitor	Dùng Visitor
Phải sửa class	Không sửa class
Vi phạm SRP	Tuân thủ SRP
Khó mở rộng	Dễ thêm chức năng

Visitor	Strategy
Thêm hành vi cho nhiều class	Thay đổi thuật toán cho một class
Double dispatch	Single dispatch
Cấu trúc object ổn định	Thuật toán thay đổi

8 Khi nào nên dùng & Bài tập thực hành

🤖 Khi nào nên dùng Visitor?

- ✓ Có nhiều loại object cố định | ✓ Thường xuyên thêm hành vi mới
- ✓ Muốn tách nghiệp vụ khỏi data model.
- ✗ Không nên dùng nếu: Thường xuyên thêm loại object mới.

📝 Bài tập thực hành

1. Tạo DiscountVisitor cho từng loại product.
2. Tạo ShippingCostVisitor.
3. Thêm loại Product mới (SubscriptionProduct).
4. Kết hợp Visitor + Composite cho OrderItem tree.



Kết luận

Visitor Pattern giúp: Thêm hành vi mới mà không sửa class, Tách biệt nghiệp vụ và dữ liệu, Tuân thủ Open/Closed Principle.

Trong Order Management System, Visitor hữu ích cho:

- Tính thuế
- Tính khuyến mãi
- Xuất báo cáo
- Kiểm tra validation